

Termin zajęć Piątek – PARZYSTY – - GODZ. 8:00	Układy cyfrowe i systemy wbudowane	
Osoby wykonujące ćwiczenie: Karol Gąsior 280913 Michał Sadowy 280884		Grupa nr: 14
Tytuł ćwiczenia: Przerzutniki		Laboratorium nr: 3
Data wykonania ćwiczenia	24-10-2025	Ocena:
Data oddania sprawozdania	21-11-2025	

1 Zadanie 1

Stworzyć licznik rewersyjny liczący w takiej kolejności – (0, 1, 3, 4, 5, 6, 2, 7). Następnie przeprowadzić symulację behawioralną oraz czasową. Całość zaprezentować na sprzęcie

1.1 Stworzenie tabeli prawdy

Tab. 1: Tabela prawdy

Nr.	Dir	Q^0_0	Q^0_1	Q^0_2	Q^1_0	Q^1_1	Q^1_2
0	0	0	0	0	0	0	1
1	0	0	0	1	0	1	1
2	0	0	1	0	1	1	1
3	0	0	1	1	1	0	0
4	0	1	0	0	1	0	1
5	0	1	0	1	1	1	0
6	0	1	1	0	0	1	0
7	0	1	1	1	0	0	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
2	1	0	1	0	1	1	0
3	1	0	1	1	0	0	1
4	1	1	0	0	0	1	1
5	1	1	0	1	1	0	0
6	1	1	1	0	1	0	1
7	1	1	1	1	0	1	0

Uwaga zamiana na wyjściu całego układu:

$Q^1_2 \rightarrow Q_0$

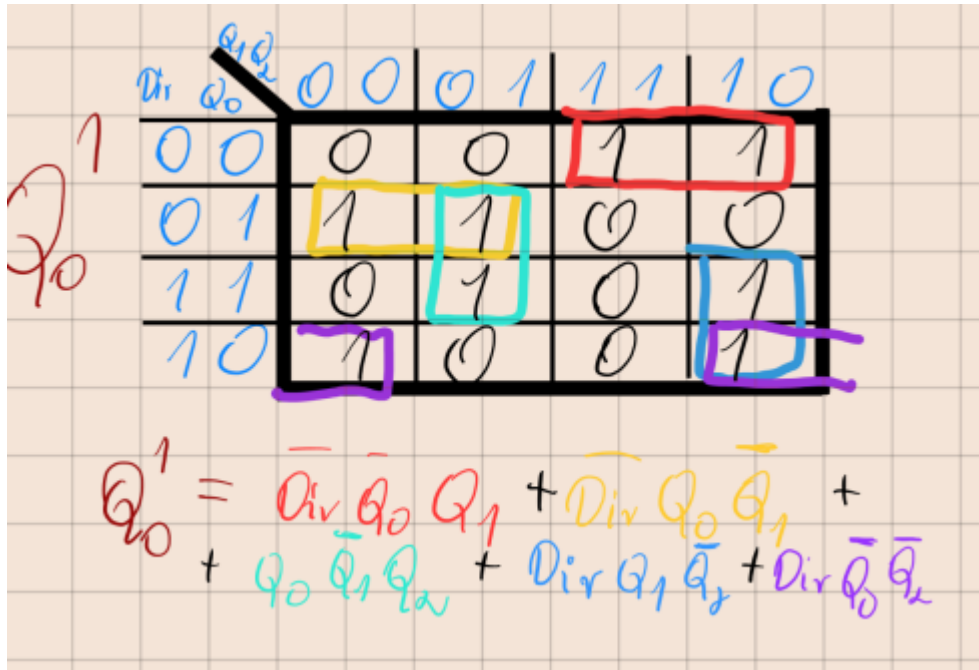
$Q^1_1 \rightarrow Q_1$

$Q^1_0 \rightarrow Q_2$

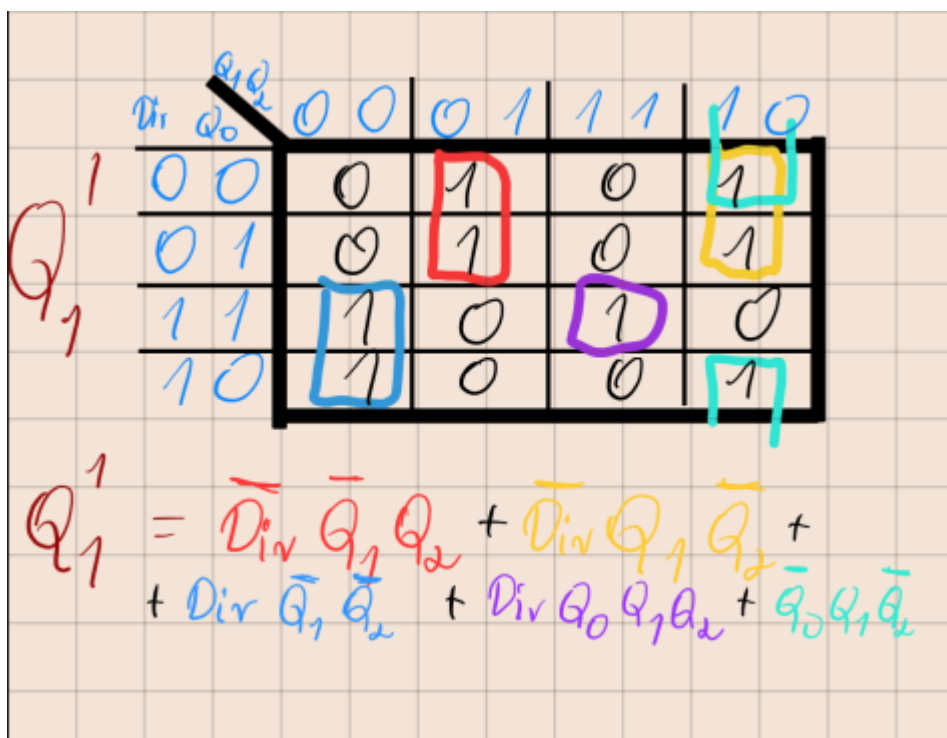
Osiągnięto poprzez Q(0:2) zamiast Q(2:0)

1.2 Zaprojektowanie układu kombinacyjnego oraz minimalizacja za pomocą siatek Karnough.

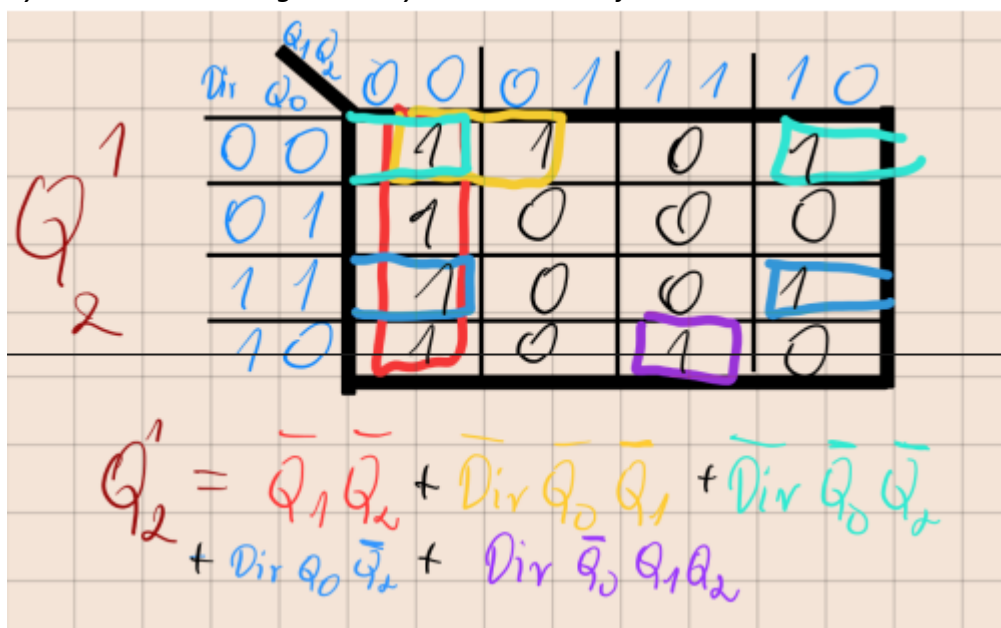
Na podstawie stworzonej tabeli prawdy, stworzono 3 siatki Karnough, odpowiednio dla kolumn Q^1_0 , Q^1_1 , Q^1_2 tabeli prawdy. Stworzono grupy implicantów a następnie otrzymane funkcje zminimalizowano. Rys.1 do Rys. 3 przedstawiają odpowiedni siatki Karnough dla kolejnych sygnałów wyjściowych oraz minimalizację ich funkcji.



Rys. 1: Siatka Karnough oraz wynik minimalizacji dla Q^1_0



Rys. 2: Siatka Karnaugh oraz wynik minimalizacji dla Q_1^1



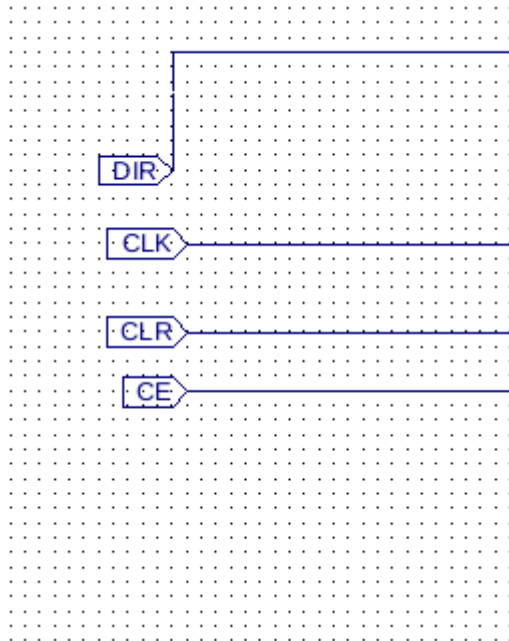
Rys. 3: Siatka Karnaugh oraz wynik minimalizacji dla Q_2^1

1.3 Stworzenie schematu logicznego w ISE

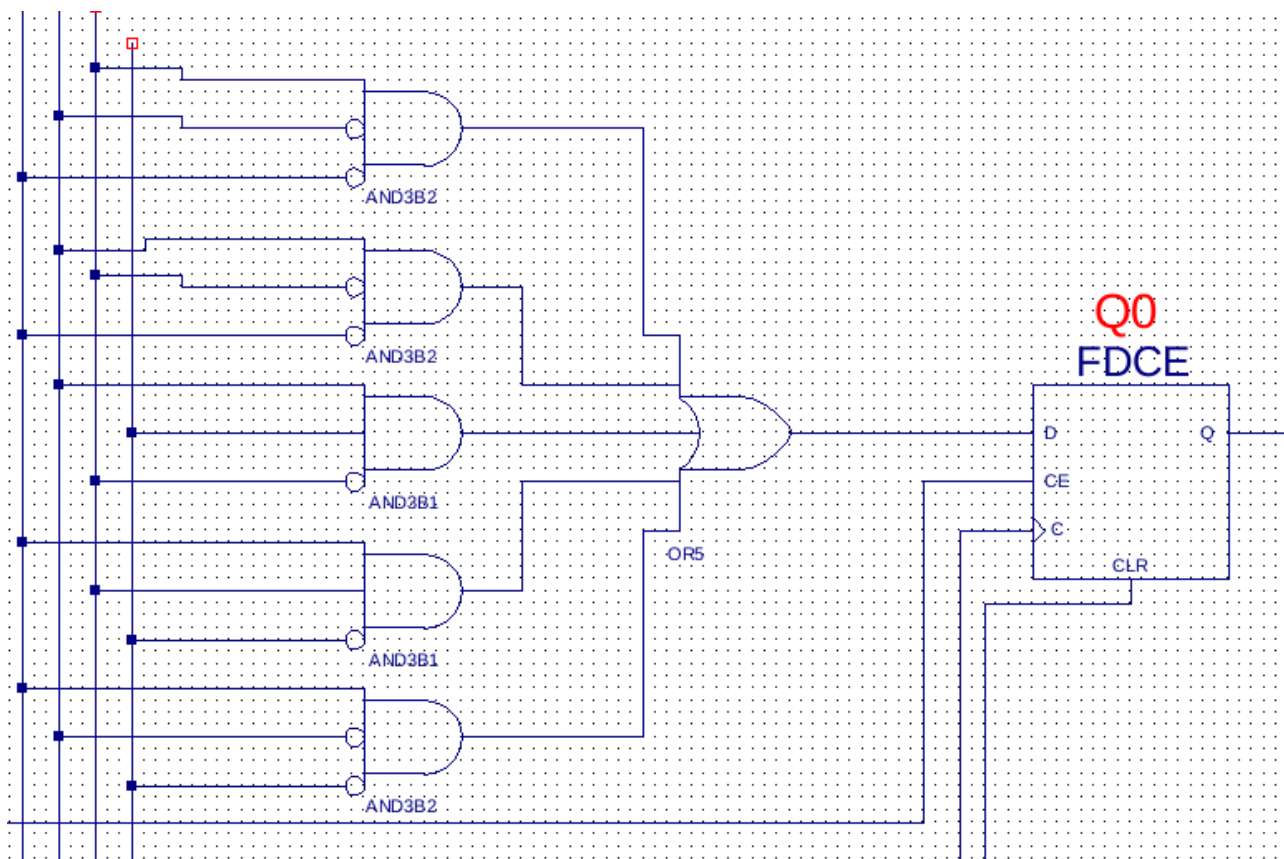
Na podstawie wyznaczonych funkcji wyjść stworzono schemat logiczny w ISE (plik *.sch). Najpierw stworzono cztery wejścia i nazwano je DIR, CLK, CLR, CE (Rys. 5), następnie dodano

kolejne bramki oraz połączono je tak, aby stworzone z nich układy logiczne odpowiadały odpowiednim funkcjom wyjścia. Rys. 5 do Rys. 7 przedstawiają schematy tych układów. Na koniec stworzono dwu-bitową magistralę wyjściową i odpowiednio podłączono do niej sygnały wyjściowe z w/w układów reprezentujących funkcję wyjściowe.

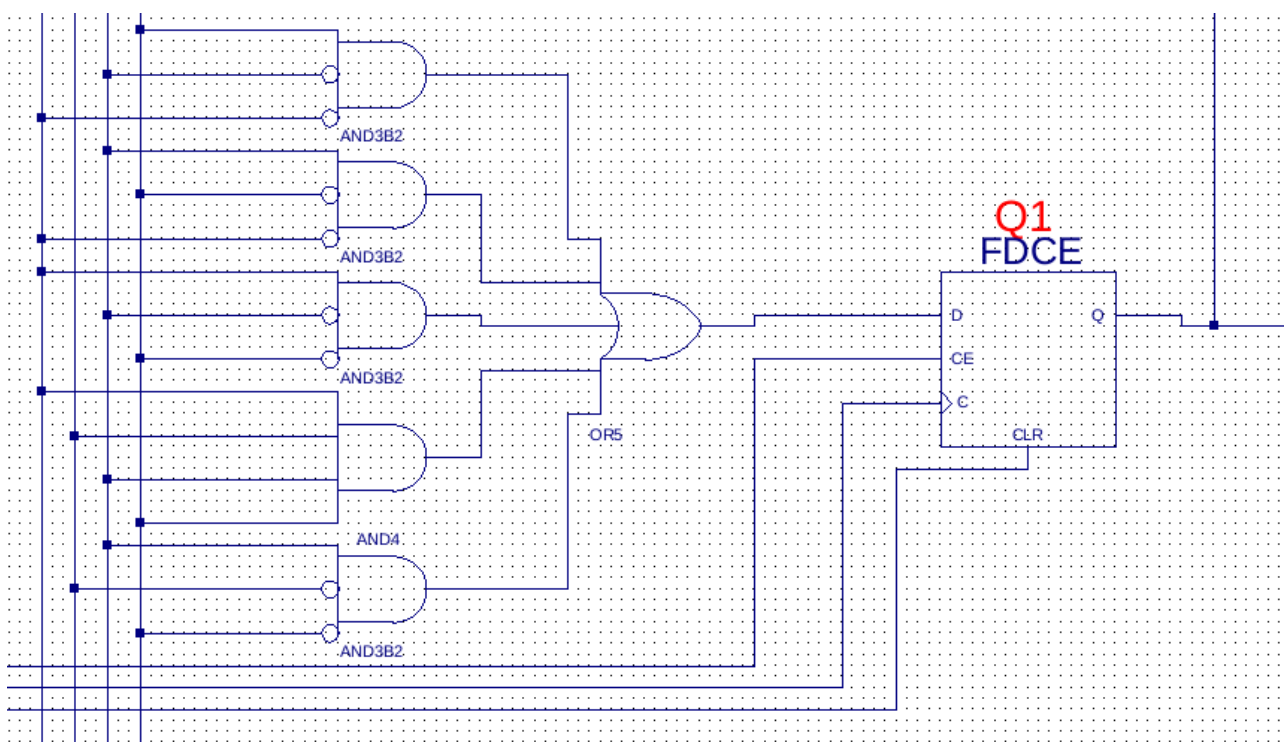
Gotowy schemat logiczny sprawdzono za pomocą przycisku *Check Schematics*.



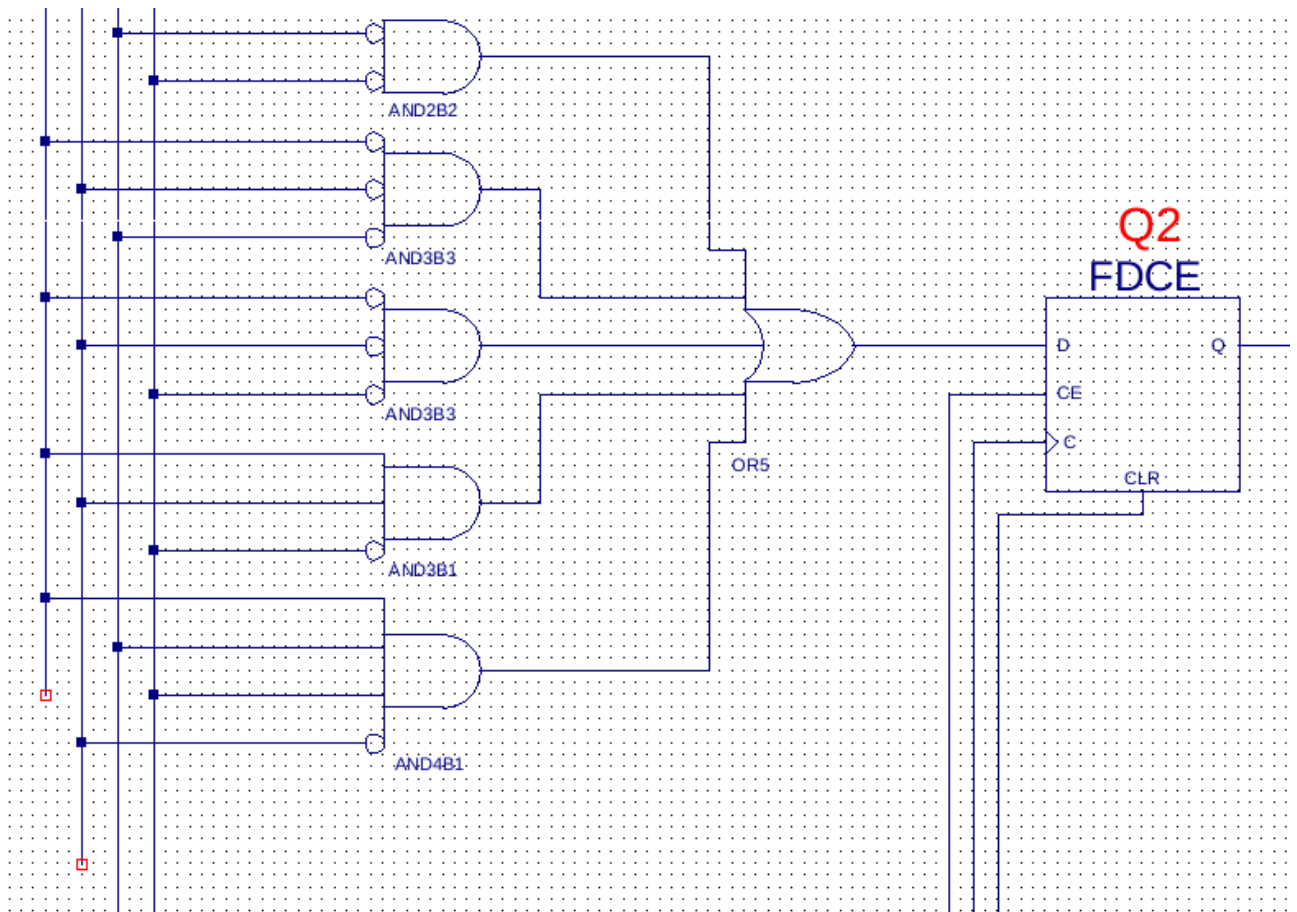
Rys. 4: Punkty wejściowe



Rys. 5: Układ logiczny odpowiadający funkcji wyjścia dla Q0



Rys. 6: Układ logiczny odpowiadający funkcji wyjścia dla Q2



Rys. 7: Układ logiczny odpowiadający funkcji wyjścia dla y1

1.4 Symulacja behawioralna i czasowa

Pierwszym krokiem do przygotowania symulacji behawioralnej jest wygenerowanie pliku VHDL, który będzie opisywał zachowanie sygnałów wejściowych. Na Rys. 10 przedstawiono fragment wygenerowanego pliku z wstawionym kodem, który definiuje zachowanie sygnałów wejściowych. Na Rys. 11 przedstawiono wynik symulacji behawioralnej. Na Rys. 12 przedstawiono output z symulacji czasowej. Czas propagacji sygnału wynosi 10 ns.

```

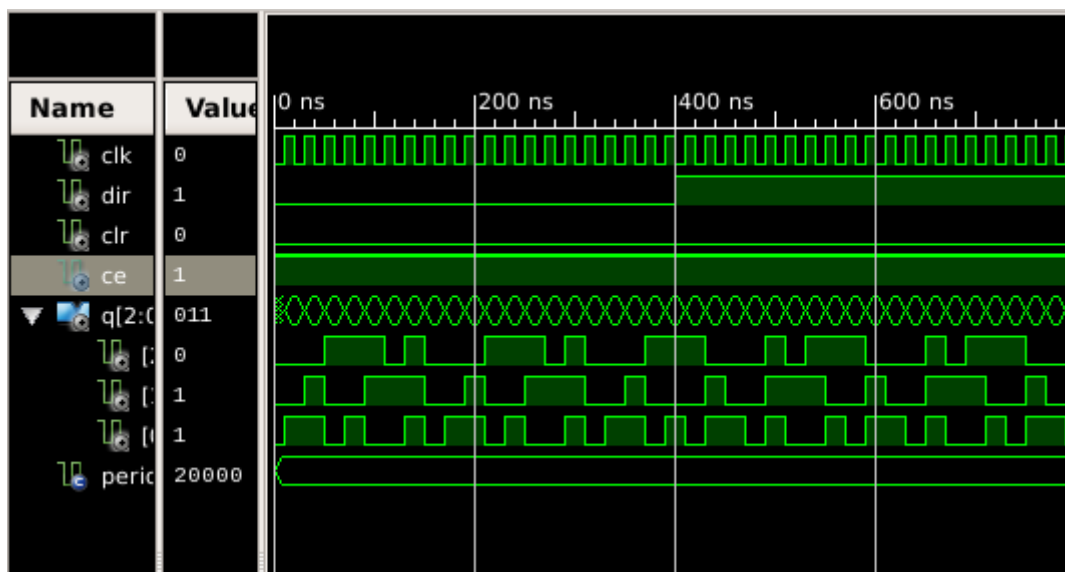
constant period : DELAY_LENGTH := 1us / 50;
BEGIN

    UUT: przerzutnik PORT MAP (
        CLK => CLK,
        DIR => DIR,
        CLR => CLR,
        CE => CE,
        Q => Q
    );

    DIR <= '1' after 20 * period;
    CLK <= not CLK after period/2;

```

Rys. 8: Fragment listingu pliku VHDL opisującego zachowanie sygnałów



Rys. 9 Output z symulacji behawioralnej

1.5 Implementacja

Implementacja stworzonego układu logicznego na płycie testowej wymaga powiązania stworzonych magistral z rzeczywistymi elementami tej płyty. Odbywa się to za pośrednictwem pliku *.ucf. Gotowy plik uzyskano od prowadzącego laboratorium. Rys. 11 przedstawia zmiany wprowadzone w pliku, polegające na ustawieniu odpowiednich sygnałów CLK i period oraz pod odpowiednimi przyciskami skonfigurowano sygnały CE, DIR i CLK. Dodatkowo skonfigurowano odpowiednie sygnały wyjściowe widoczne na rys. 12


```

5  # Clocks
6  NET "CLK" LOC = "P7" | BUFG = CLK;
7  NET "CLK" PERIOD = 5ms HIGH 50%;
8
9  #NET "Clk_XT" LOC = "P5" | BUFG = CLK;
10 #NET "Clk_XT" PERIOD = 500ns HIGH 50%;
11
12 # Keys
13 NET "CE" LOC = "P42";
14 NET "DIR" LOC = "P40";
15 NET "CLR" LOC = "P43";
16 #NET "Key<3>" LOC = "P38";
17 #NET "Key<4>" LOC = "P37";
18 #NET "Key<5>" LOC = "P36"; # shared with ROT_A
19 #NET "Key<6>" LOC = "P24"; # shared with ROT_B
20 #NET "Key<7>" LOC = "P39"; # GSR
21

```

Rys. 11: Fragment pliku *.ucf definiującego sygnały wejściowe.

```

41 # DISPL. 7-SEG
42 #NET "D7S_D<0>" LOC = "P8" | SLEW = "SLOW";
43 #NET "D7S_D<1>" LOC = "P6" | SLEW = "SLOW";
44 #NET "D7S_D<2>" LOC = "P4" | SLEW = "SLOW";
45 #NET "D7S_D<3>" LOC = "P9" | SLEW = "SLOW";
46 NET "Y<0>" LOC = "P12"; # Seg. A; shared with LED<10>
47 NET "Y<1>" LOC = "P13"; # Seg. B; shared with LED<8>
48 NET "Y<2>" LOC = "P22"; # Seg. C; shared with LED<12>
49 NET "Y<3>" LOC = "P19"; # Seg. D; shared with LED<14>
50 NET "Y<4>" LOC = "P14"; # Seg. E; shared with LED<15>
51 NET "Y<5>" LOC = "P11"; # Seg. F; shared with LED<9>
52 NET "Y<6>" LOC = "P20"; # Seg. G; shared with LED<13>
53 #NET "D7S_S<7>" LOC = "P18"; # Seg. DP; shared with LED<11>
54

```

Rys. 12: Fragment pliku *.ucf definiującego wyświetlacz segmentowy.

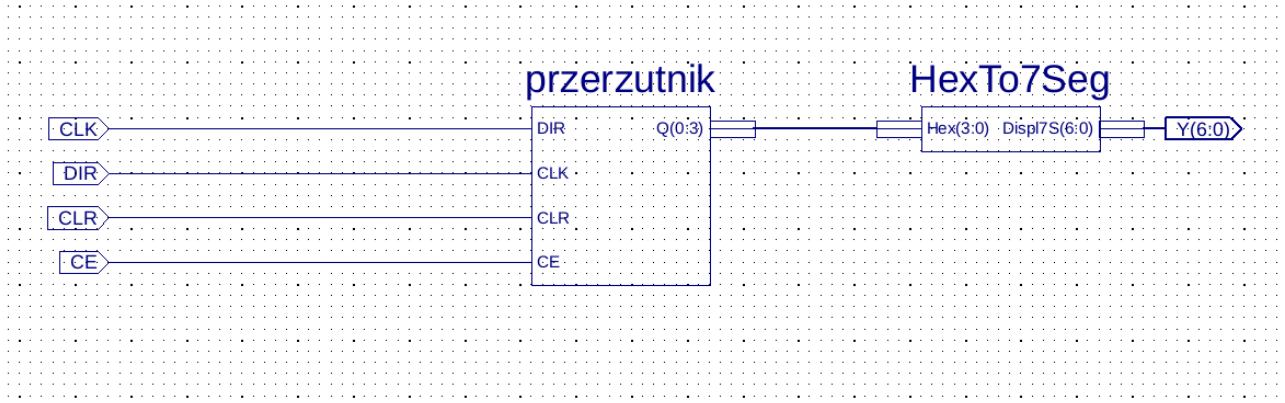
1.6 Programowanie układu, testowanie

Zgodnie z instrukcją dostarczoną przez prowadzącego wygenerowane pliki oraz zaprogramowano płytę ZL-9572, a następnie zadając kolejne kombinacje przetestowano układ. Układ działał inaczej niż powinien, a mianowicie liczby były reprezentowane odwrotnie (Zapalały się inne niż powinny).

Przeprowadzone testy potwierdziły, że zaprojektowany układ działał poprawnie, co zostało sprawdzone przez prowadzącego.

2 Zadanie 2, stworzenie projektu hierarchicznego oraz wykorzystanie wyświetlacza segmentowego.

Zgodnie z instrukcją laboratoryjną z układ stworzony w zadaniu 1 przekształcono na submoduł i stworzono jego symbol schematowy. Rys. 13 przedstawia nowo powstały schemat. Rys 14 przedstawia fragment pliku *.ucf wiążący magistralę wyjściową z wyświetlaczem segmentowym.



Rys. 13: Schemat powstały w ramach zadania drugiego

```
# DISPL. 7-SEG
#NET "D7S_D<0>" LOC = "P8" | SLEW = "SLOW";
#NET "D7S_D<1>" LOC = "P6" | SLEW = "SLOW";
#NET "D7S_D<2>" LOC = "P4" | SLEW = "SLOW";
#NET "D7S_D<3>" LOC = "P9" | SLEW = "SLOW";
NET "Y<0>" LOC = "P12"; # Seg. A; shared with LED<10>
NET "Y<1>" LOC = "P13"; # Seg. B; shared with LED<8>
NET "Y<2>" LOC = "P22"; # Seg. C; shared with LED<12>
NET "Y<3>" LOC = "P19"; # Seg. D; shared with LED<14>
NET "Y<4>" LOC = "P14"; # Seg. E; shared with LED<15>
NET "Y<5>" LOC = "P11"; # Seg. F; shared with LED<9>
NET "Y<6>" LOC = "P20"; # Seg. G; shared with LED<13>
#NET "D7S_S<7>" LOC = "P18"; # Seg. DP; shared with LED<11>
```

Rys. 14: Fragment pliku *.ucf odpowiadający za powiązanie magistrali Y(6:0) z wyświetlaczem segmentowym

Również w tym przypadku zaprogramowano płytę testową oraz sprawdzono fizyczne działanie układu. Układ działał poprawnie.

3 Podsumowanie

W trakcie laboratorium zapoznano się z podstawami pracy w środowisku ISE 14.7 stworzono prosty licznik rewersyjny następnie przeprowadzono symulację behawioralną oraz czasową. W kolejnym kroku wygenerowano plik dla programatora. Programator poprzez JTAG zaprogramował płytę ZL-9572. Zwieńczeniem laboratorium było przeprowadzanie testów poprawności zaprojektowania, implementacji oraz zaprogramowania płyty. Całość działała poprawnie.